

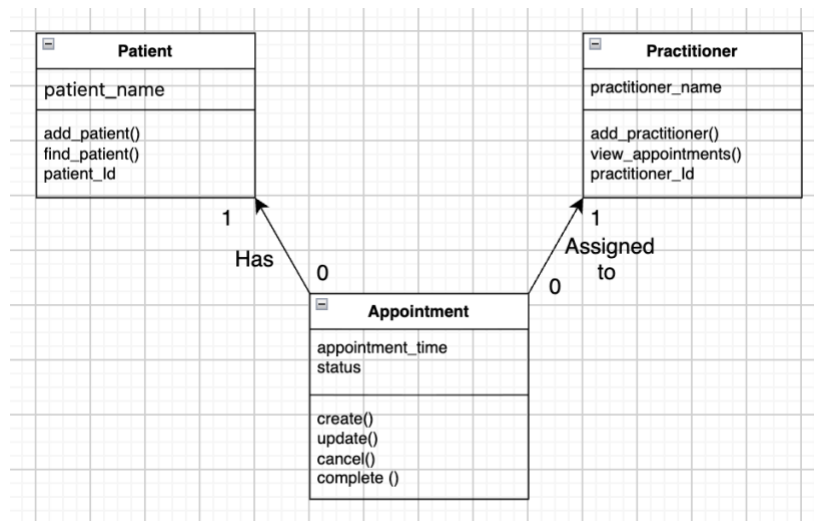
Assignment 2-Case Study

Stage 4 Lab Activities

Implementing the SmartCare Domain Layer

DESIGN FIRST -> AI PAIR PROGRAMMING -> REVIEW -> VERIFY | 1hour

A - Revisit Approved UML



Confirm responsibilities, attributes and relationships before coding.

My UML diagram has three classes. Patient, Practitioner and Appointment. The patient and practitioner classes store the basic details. Appointment is the class that links one patient to one practitioner and it keeps track of the time and status.

B - Implement Patient: AI OFF

Implement Patient with type hints and basic validation.

Clas patient is the blueprint for making patient objects. `_init_` runs automatically when a new patient is created and sets up the objects starting data attributes.

`Self.__patient_id` and `self.__patient_name` are the data attributes. The double underscore is hiding attributes which means that the code outside the class can't reach in and change them directly.

The if/else check on `patient_name` is that if any empty name is passed in then it prints an error message and fails back to being unknown and doesn't accept unknown data that is not correct.

`get_patient_id()` and `get_practitioner_name()`. Lets the outside cde read the private attributes without being able to change it.

Set_patient_name() is the only way to change the name from outside the class and it runs the same empty-string check before allowing the change. It does a true or false Boolean flag s the code knows whether the change actually happened.

C - Implement Practitioner: AI OFF

Implement Practitioner with identifier, name and specialty; no database logic.

The patient __init__ sets up _practitioner_id, _practitioner_name and _specialty.

Get_ methods read them

set_specialty() change the specialty with the same if/else empty-check and True/False return.

No new concepts – which it is just patient pattern applied to a second class

D - Implement Appointment: AI ON

Give AI the approved Appointment UML, business rules and explicit constraints. Ask it to implement only Appointment and agreed enum/exception.

_status = “SCHEDULED” – it is the status that is stored as a plain private string which is set automatically when the appointment is created. It is double-underscore which nothing outside the class can just overwrite it.

Complete() and cancel() they are the only two ways to change the status. Each use a simple if/else check. If the appointment is currently SCHEDULED, the change is allowed and the method returns True otherwise it prints a message explaining why and returns False. It is the same True/False flag pattern used in set_patient_name().

E - Review Generated Code

Check model consistency, unsupported features, public state mutation, unnecessary inheritance, invented dependencies and error handling.

1. It gave Appointment a set_status() method that just overwrote the status with whatever was passed in. We removed it so status can only change through cancel()/complete().
2. It added a NotificationManager the brief never asked for which was deleted
3. It made Appointment inherit from a generic “Record” class using the class Appointment(Record). An appointment isn’t a specialized kind of record so I removed the inheritance and just kept _patient and _practitioner as ordinary attributes instead.

F - Manual Behaviour Checks

Create valid objects, test invalid input, cancel a scheduled appointment and attempt an illegal repeated transition.

G - Refactor

Remove unnecessary code and make implementation simpler and design-consistent.

H - AI Engineering Log

Record prompt, generated contribution, decisions and verification evidence.

Suggested AI prompt

Act as a Python pair programmer. Implement only the Appointment class from the approved SmartCare UML. Use type hints and an AppointmentStatus enum. Cancelled appointments remain as objects. Do not add database, UI, notification or service classes. Protect status transitions and explain any decision not directly visible in the UML.

Reflection

Which AI-generated part did you modify or reject? Why? How did the approved design constrain the AI?

Rejected the unchecked `set_status()` method, the `NotificationManager`, and the inheritance from a record class. It was rejected as none of them were in the approved UML. The extra code didn't relate to the UML diagram.

The UML was like a check list and most things AI added was not on the checklist. The notifications it stated and extra base class were removed as the original code was compared to the generated code.

Some code was made with the help of AI.